

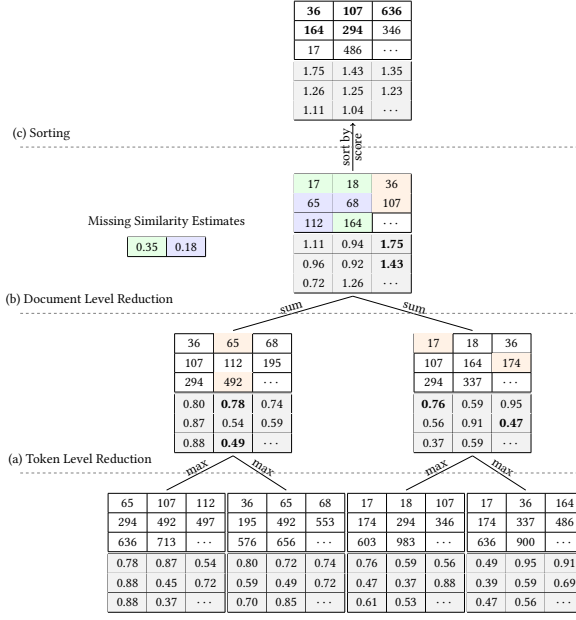


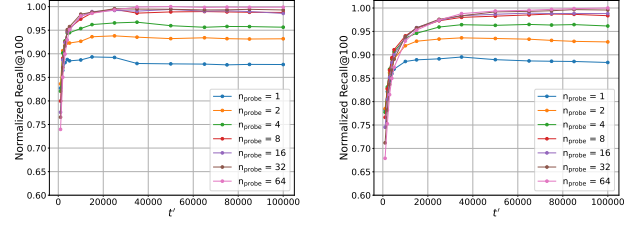
Figure 5: WARP’s scoring phase: (a) In token-level reduction, strides are max-reduced. (b) In document-level reduction, values are summed, accounting for missing similarity estimates. (c) Scores are sorted, yielding the top- k results.

To study the effects of n_{probe} and t' , we analyze the normalized Recall@100¹¹ as a function of t' for $n_{\text{probe}} \in \{1, 2, 4, 8, 16, 32, 64\}$ across four development datasets of increasing size: BEIR NFCorpus, BEIR Quora, LoTTE Lifestyle, and LoTTE Pooled. For further details on the datasets, please refer to Table 1. Figure 6 visualizes the results of our analysis. We observe a consistent pattern across all evaluated datasets, namely substantial improvements as n_{probe} increases from 1 to 16 (i.e., 1, 2, 4, 8, 16), followed by only marginal gains in Recall@100 beyond that. A notable exception is BEIR NFCorpus, where we still observe significant improvement when increasing from $n_{\text{probe}} = 16$ to $n_{\text{probe}} = 32$. We hypothesize that this is due to the small number of embeddings per cluster in NFCorpus, limiting the number of scores available for aggregation. Consequently, we conclude that setting $n_{\text{probe}} = 32$ strikes a good balance between end-to-end latency and retrieval quality.

In general, we find that WARP is highly robust to variations in t' . However, smaller datasets, such as NFCorpus, appear to benefit from a smaller t' , while larger datasets perform better with a larger t' . Empirically, we find that setting t' proportional to the square root of the dataset size consistently yields strong results across all datasets. Moreover, increasing t' beyond a certain point no longer improves recall, leading us to bound t' by a maximum value, t'_{max} .

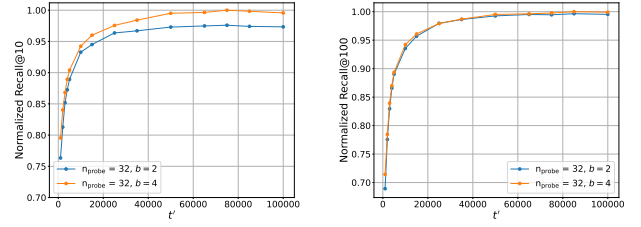
Next, we aim to quantify the effect of b on the retrieval quality of WARP, as shown in Figure 7. To do this, we compute the nRecall@ k for $n_{\text{probe}} = 32$ and $k \in \{10, 100\}$ using two datasets: LoTTE Science and LoTTE Pooled.

¹¹The normalized Recall@ k is calculated by dividing Recall@ k by the dataset’s maximum, effectively scaling values between 0 and 1 to ensure comparability across datasets.



(a) LoTTE Science (Dev Set) (b) LoTTE Pooled (Dev Set)

Figure 6: nRecall@100 as a function of t' and n_{probe}



(a) LoTTE Pooled (Dev Set), nRecall@10 (b) LoTTE Pooled (Dev Set), nRecall@100

Figure 7: nRecall@ k as a function of t' and b

Dataset		Dev		Test	
		#Queries	#Passages	#Queries	#Passages
BeIR [20]	NFCORPUS	324	3.6K	323	3.6K
	SciFact	–	–	300	5.2K
	SCIDOCS	–	–	1,000	25.7K
	Quora	5,000	522.9K	10,000	522.9K
	FiQA-2018	500	57.6K	648	57.6K
	Touché-2020	–	–	49	382.5K
LoTTE [19]	Lifestyle	417	268.9K	661	119.5K
	Recreation	563	263.0K	924	167.0K
	Writing	497	277.1K	1,071	200.0K
	Technology	916	1.3M	596	638.5K
	Science	538	343.6K	617	1.7M
	Pooled	2,931	2.4M	3,869	2.8M

Table 1: Datasets used for evaluating XTR_{base}/WARP performance. The evaluation includes 6 datasets from BEIR [20] and 6 from LoTTE [19].

Our results show a significant improvement in retrieval performance when increasing b from 2 to 4, particularly for smaller values of k . For larger values of k , the difference in performance diminishes, particularly for the LoTTE Pooled dataset.

5 Evaluation

We now evaluate WARP on six datasets from BEIR [20] and six datasets from LoTTE [19] listed in Table 1. We use servers with 28 Intel Xeon Gold 6132 @ 2.6 GHz CPU cores¹² and 500 GB

¹²Each core has 2 threads for a total of 56 threads.

RAM. The servers have two NUMA sockets with roughly 92 ns intra-socket memory latency, 142 ns inter-socket memory latency, 72 GBps intra-socket memory bandwidth, and 33 GBps inter-socket memory bandwidth.

When measuring latency for end-to-end results, we compute the average latency of all queries and report the minimum average latency across three trials. For other results, we describe the specific measurement procedure in the relevant section. We measure latency on an otherwise idle machine. XTR’s token retrieval stage does not benefit from GPU acceleration due to its use of ScaNN [5], specifically designed for single-threaded¹³ use on x86 processors with AVX2 support. Therefore, unless otherwise specified, all measurements are performed on the CPU using a single thread¹⁴.

5.1 End-to-End Results

Recent studies, such as those in [14], demonstrate that dense multi-vector retrieval systems deliver near-exhaustive search performance while significantly reducing computational costs. In contrast, conventional re-ranking, though generally faster, often underperforms multi-vector retrieval systems in terms of effectiveness. As a result, we limit our evaluation to a comparison with the XTR/ScaNN baseline and leave a more extensive evaluation of re-ranking or exhaustive approaches to future work.

Table 2 presents our results on LoTTE. XTR_{base}/WARP outperforms the optimized XTR_{base}/ScaNN implementation in terms of Success@5, while significantly reducing end-to-end latency, with speedups ranging from 4.6x on LoTTE Lifestyle to 12.8x on LoTTE Pooled. As shown in Table 3, we observe a similar trend with the evaluation of nDCG@10 on the six BEIR [20] datasets. XTR_{base}/WARP achieves speedups of 2.7x-6x over XTR_{base}/ScaNN with a slight gain in nDCG@10. Likewise, we find improvements of Recall@100 on BEIR with substantial gains in end-to-end latency, but we omit them here due to space constraints.

In comparison, EMVB [15] reports a notable 2.9x speedup in retrieval latency (142ms vs. 411ms) over ColBERTv2/PLAID on the LoTTE Pooled development set. By contrast, our method achieves a 4.3x speedup (95ms vs. 405ms), albeit under different hardware settings and encoder models. Importantly, we view WARP and EMVB as largely orthogonal approaches, suggesting that future work could explore integrating SIMD-based acceleration into WARP.

5.2 Scalability

We now assess WARP’s scalability in relation to both dataset size and the degree of parallelism. To study the effect of the dataset size on WARP’s performance, we evaluate its latency across development datasets of varying sizes: BEIR NFCorpus, BEIR Quora, LoTTE Science, LoTTE Technology, and LoTTE Pooled (Table 1). Figure 8a plots the latency of different configurations versus the size of the dataset, measured in the number of document token embeddings. Our results confirm that WARP’s latency generally scales with the square root of the dataset size — this is intuitive, as the number of clusters is by design proportional to the square root of the dataset size.

¹³As of the recently released version 1.3.0, ScaNN supports multi-threaded search via the `search_batched_parallel` function.

¹⁴We believe WARP’s design to naturally carry over to a GPU implementation, though this is left for future work.

	Lifestyle	Recreation	Writing	Technology	Science	Pooled	Avg.
BM25	63.8	56.5	60.3	41.8	32.7	48.3	50.6
ColBERT	80.2	68.5	74.7	61.9	53.6	67.3	67.7
GTR _{base}	82.0	65.7	74.1	58.1	49.8	65.0	65.8
XTR/ScaNN ¹⁵	83.5 (333.6)	69.6 (400.2)	78.0 (378.0)	63.9 (742.5)	55.3 (1827.6)	68.4 (2156.3)	69.8
XTR/WARP	83.5 (73.1)	69.5 (72.4)	78.6 (73.6)	64.6 (96.4)	56.1 (156.4)	69.3 (171.3)	70.3
Splade _{v2} [⚡]	82.3	69.0	77.1	62.4	55.4	68.9	69.2
ColBERT _{v2} [⚡]	84.7	72.3	80.1	66.1	56.7	71.6	71.9
GTR _{xcd}	87.4	78.0	<u>83.9</u>	69.5	60.0	76.0	75.8
XTR _{xcd}	<u>89.1</u>	<u>79.3</u>	83.3	<u>73.7</u>	<u>60.8</u>	<u>77.3</u>	<u>77.3</u>

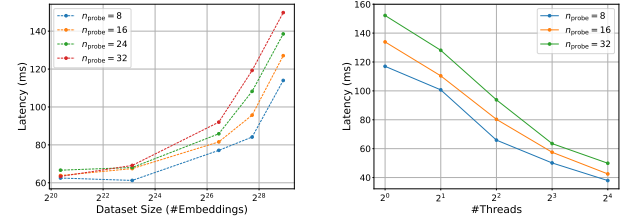
[⚡]: cross-encoder distillation [⚡]: model-based hard negatives

Table 2: Success@5 on LoTTE. Numbers in parentheses show average latency (milliseconds), with the final column displaying the average score across the datasets. Both XTR/ScaNN and WARP use the XTR_{base} model. XTR/ScaNN uses $k' = 40\,000$ and WARP uses $n_{\text{probe}} = 32$.

	NFCorpus	SciFact	SCIDocs	FiQA-2018	Touché-2020	Quora	Avg.
BM25	32.5	66.5	15.8	23.6	36.7	78.9	42.3
ColBERT	30.5	67.1	14.5	31.7	20.2	85.4	41.6
GTR _{base}	30.8	60.0	14.9	34.9	21.5	88.1	41.7
XTR/ScaNN ¹⁵	33.5 (158.1)	69.6 (309.7)	14.3 (297.3)	34.1 (338.2)	31.2 (560.2)	86.0 (411.2)	44.8
XTR/WARP	33.5 (58.0)	70.5 (64.3)	15.2 (66.1)	34.2 (70.7)	30.5 (94.8)	86.2 (67.6)	45.0
Splade _{v2} [⚡]	33.4	69.3	15.8	33.6	27.2	83.8	43.8
ColBERT _{v2} [⚡]	33.8	69.3	15.4	35.6	26.3	85.2	44.3
GTR _{xcd}	34.2	66.2	16.1	46.7	23.3	89.2	45.9
XTR _{xcd}	<u>35.3</u>	<u>74.3</u>	<u>17.1</u>	43.8	30.9	88.1	<u>48.3</u>

[⚡]: cross-encoder distillation [⚡]: model-based hard negatives

Table 3: nDCG@10 on BEIR. Numbers in parentheses show average latency (milliseconds), with the final column displaying the average score across the datasets. Both XTR/ScaNN and WARP use the XTR_{base} model. XTR/ScaNN uses $k' = 40\,000$ and WARP uses $n_{\text{probe}} = 32$.



(a) End-to-end latency vs dataset size (measured in #embeddings)

(b) End-to-end latency for varying n_{probe}

Figure 8: WARP’s scaling behavior with respect to dataset size and the number of available CPU threads

A key advantage of WARP over the reference implementation is its ability to leverage multi-threading. Figure 8b illustrates WARP’s performance on the LoTTE Pooled development set, showing how the number of CPU threads impacts performance for different values of n_{probe} . Our results indicate that WARP effectively parallelizes across multiple threads, achieving a speedup of 3.1x for $n_{\text{probe}} = 32$ with 16 threads. We refer to A.1, and in particular Figure 10, for a more detailed breakdown of WARP’s multi-threaded latency.

¹⁵Results differ marginally from [11] due to float32 vs. bfloat16 encoder inference.

5.3 Memory Footprint

Dataset	# Tokens	XTR				
		Index Size (GiB)				
		BruteForce	FAISS	ScaNN	WARP _(b=2)	WARP _(b=4)
BeIR [20]	NFCORPUS	1.35M	0.65	0.06	0.18	0.06
	SciFact	1.87M	0.91	0.08	0.25	0.07
	SCIDOCS	6.27M	3.04	0.28	0.82	0.24
	Quora	9.12M	4.43	0.43	1.21	0.35
	FiQA-2018	10.23M	4.95	0.46	1.34	0.38
	Touché-2020	92.64M	45.01	4.28	12.22	3.40
LoTTE [19]	Lifestyle	23.71M	11.51	1.08	3.12	0.88
	Recreation	30.04M	14.59	1.38	3.96	1.11
	Writing	32.21M	15.64	1.48	4.25	1.19
	Technology	131.92M	64.12	6.13	17.44	4.83
	Science	442.15M	214.93	20.57	58.46	16.07
	Pooled	660.04M	320.88	30.74	87.30	23.88
Total	–	1.44B	700.66	66.98	190.52	52.48

Table 4: Comparison of index sizes for the datasets. Note that PLAID’s memory usage is effectively identical to WARP’s, only slightly larger.

WARP’s primary advantage lies in its significant reduction in latency, though its benefits extend beyond speed alone. By adopting a ColBERTv2/PLAID-like approach for compression, WARP’s advantage over XTR also translates into a reduction in index size. This decrease in memory requirements broadens deployment options, particularly in resource-constrained environments. Table 4 compares index sizes across all evaluated test datasets. WARP_(b=4) demonstrates a substantially smaller index size compared to the BruteForce and ScaNN variants, providing a 7.3x and 2x reduction in memory footprint, respectively. Although the indexes generated by the FAISS implementation are marginally smaller, this comes at the cost of substantially reduced quality and latency. Notably, WARP_(b=2) outperforms the FAISS implementation in terms of quality with an even smaller index size¹⁶.

6 Conclusion

We present WARP, a highly optimized engine for multi-vector retrieval built upon ColBERTv2/PLAID and the XTR framework. WARP addresses key inefficiencies in existing systems through three major innovations: (1) WARP_{SELECT} for dynamic missing similarity imputation; (2) implicit decompression during retrieval; and (3) a streamlined two-stage reduction using dedicated C++ kernels. Together, these optimizations culminate in substantial performance gains, including a 41x speedup over XTR on LoTTE Pooled – cutting latency from over 6 seconds to just 171ms in single-threaded execution – and a 3x reduction in latency compared to ColBERTv2/PLAID, all without compromising retrieval quality. WARP also scales efficiently with increased thread count and offers a significantly reduced memory footprint, making it well-suited for deployment in resource-constrained environments.

Future work may incorporate techniques such as SIMD and GPU acceleration, more lightweight query encoding, and end-to-end training with WARP to better align model optimization with its retrieval process.

¹⁶ Additionally, WARP reduces memory requirements compared to PLAID as it no longer requires storing a mapping from document ID to centroids/embeddings.

References

- [1] Thibault Formal, Stéphane Clinchant, Hervé Déjean, and Carlos Lassance. 2024. Splate: sparse late interaction retrieval. (2024). arXiv: 2404.13950 [cs. IR].
- [2] Luyu Gao, Zhuyun Dai, and Jamie Callan. 2021. Coil: revisit exact lexical match in information retrieval with contextualized inverted list. (2021). <https://arxiv.org/abs/2104.07186> arXiv: 2104.07186 [cs. IR].
- [3] Stanford Future Data Systems Research Group. 2024. colbert-ir/colbertv2.0. <https://huggingface.co/colbert-ir/colbertv2.0>. (2024).
- [4] Stanford Future Data Systems Research Group. 2024. ColBERTv2/PLAID (Code). <https://github.com/stanford-futuredata/ColBERT>. (2024).
- [5] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. (2020). <https://arxiv.org/abs/1908.10396> arXiv: 1908.10396 [cs. LG].
- [6] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33, 1, 117–128. doi: 10.1109/TPAMI.2010.57.
- [7] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, (Eds.), Association for Computational Linguistics, Online, (Nov. 2020), 6769–6781. doi: 10.18653/v1/2020.emnlp-main.550.
- [8] Omar Khattab and Matei Zaharia. 2020. Colbert: efficient and effective passage search via contextualized late interaction over BERT. *CoRR*, abs/2004.12832. <https://arxiv.org/abs/2004.12832> arXiv: 2004.12832.
- [9] Ron Kohavi, Alex Deng, Brian Frasca, Toby Walker, Ya Xu, and Nils Pohlmann. 2013. Online controlled experiments at large scale. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '13)*. Association for Computing Machinery, Chicago, Illinois, USA, 1168–1176. ISBN: 9781450321747. doi: 10.1145/2487575.2488217.
- [10] Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftekhar Naim, Ming-Wei Chang, and Vincent Y. Zhao. 2024. google/xtr-base-en. <https://huggingface.co/google/xtr-base-en>. (2024).
- [11] Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftekhar Naim, Ming-Wei Chang, and Vincent Y. Zhao. 2024. Rethinking the role of token retrieval in multi-vector retrieval. (2024). arXiv: 2304.01982 [cs. CL].
- [12] Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftekhar Naim, Ming-Wei Chang, and Vincent Y. Zhao. 2024. XTR: Rethinking the Role of Token Retrieval in Multi-Vector Retrieval (Code). <https://github.com/google-deepmind/xtr>. (2024).
- [13] Minghan Li, Sheng-Chieh Lin, Barlas Oguz, Asish Ghoshal, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xilun Chen. 2022. Citadel: conditional token interaction via dynamic lexical routing for efficient and effective multi-vector retrieval. (2022). <https://arxiv.org/abs/2211.10411> arXiv: 2211.10411 [cs. IR].
- [14] Sean MacAvaney and Nicola Tonello. 2024. A reproducibility study of plaid. *arXiv preprint arXiv:2404.14989*.
- [15] Franco Maria Nardini, Cosimo Rulli, and Rossano Venturini. 2024. Efficient multi-vector dense retrieval using bit vectors. (2024). arXiv: 2404.02805 [cs. IR].
- [16] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2023. Exploring the limits of transfer learning with a unified text-to-text transformer. (2023). <https://arxiv.org/abs/1910.10683> arXiv: 1910.10683 [cs. LG].
- [17] Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3, 4, 333–389. doi: 10.1561/15000000019.
- [18] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. 2022. Plaid: an efficient engine for late interaction retrieval. (2022). arXiv: 2205.09707 [cs. IR].
- [19] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2021. Colbertv2: effective and efficient retrieval via lightweight late interaction. *CoRR*, abs/2112.01488. <https://arxiv.org/abs/2112.01488> arXiv: 2112.01488.
- [20] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. Beir: a heterogeneous benchmark for zero-shot evaluation of information retrieval models. (2021). <https://arxiv.org/abs/2104.08663> arXiv: 2104.08663 [cs. IR].
- [21] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate nearest neighbor negative contrastive learning for dense text retrieval. (2020). <https://arxiv.org/abs/2007.00808> arXiv: 2007.00808 [cs. IR].
- [22] Jingtao Zhan, Jiaxin Mao, Yiqun Liu, Jiafeng Guo, Min Zhang, and Shaoping Ma. 2021. Optimizing dense retrieval model training with hard negatives. (2021). <https://arxiv.org/abs/2104.08051> arXiv: 2104.08051 [cs. IR].